

Context Precision, Worked Out by Hand

The retrieval-side RAGAS metric that asks whether the *relevant* chunks were ranked near the *top* — a rank-aware precision that rewards putting signal before noise. Computed position by position.

What this document does. It defines context precision as a rank-weighted average of precision-at- k over the positions where relevant chunks appear, computes it on a five-chunk retrieved list, and explains why ranking (not just presence) is what it scores. Numbers from Appendix A.

One of four RAGAS metrics: context precision (#27, “were relevant chunks ranked high?”) pairs with context recall (#28, “were all needed chunks retrieved?”) to diagnose the *retriever*.

1 What it measures

After retrieval, the context handed to the LLM is an *ordered* list of chunks. Context precision asks: are the relevant chunks concentrated at the top, or scattered among noise? High precision means the signal the LLM reads first is actually useful; low precision means relevant chunks are buried beneath irrelevant ones.

Intuition

Order matters because LLMs are position-sensitive: chunks near the top of the context get more attention, and a relevant chunk at rank 1 helps far more than the same chunk at rank 5 behind four distractors. Context precision is rank-aware on purpose — it does not merely ask “is the relevant chunk *present*?” (that is recall) but “is it *near the front*?” A retriever that finds the right chunk and ranks it third is worse than one that ranks it first, and this metric says so.

2 The formula

For a retrieved list of K chunks where $v_k \in \{0, 1\}$ flags whether chunk k is relevant, context precision averages precision-at- k over the *relevant* positions:

$$\text{Context Precision@}K = \frac{\sum_{k=1}^K (\text{Precision@}k \cdot v_k)}{\#\{\text{relevant chunks}\}}, \quad \text{Precision@}k = \frac{\sum_{j=1}^k v_j}{k}.$$

Precision@ k is the fraction of the top- k that is relevant; weighting by v_k means only positions *holding* a relevant chunk contribute, and dividing by the count of relevant chunks normalises to $[0, 1]$. (This is mean average precision restricted to one query — see #30.)

3 Worked by hand

A retriever returns five chunks; a judge marks each relevant (1) or not (0):

$$v = (1, 0, 1, 1, 0) \quad (\text{relevant at ranks 1, 3, 4; 3 relevant total}).$$

Step 1 — Precision@ k at each relevant position.

rank k	v_k	relevant so far	Precision@ k
1	1	1	$1/1 = 1.0000$
2	0	1	(skip: $v_2 = 0$)
3	1	2	$2/3 = 0.6667$
4	1	3	$3/4 = 0.7500$
5	0	3	(skip: $v_5 = 0$)

Step 2 — average over the 3 relevant positions.

$$\text{Context Precision} = \frac{1.0000 + 0.6667 + 0.7500}{3} = \frac{2.4167}{3} = \boxed{0.8056}.$$

A solid 0.81: the first chunk is relevant (precision 1.0), but an irrelevant chunk at rank 2 dilutes the precision of the relevant chunks that follow.

Mini-example · same chunks, better order $\Rightarrow$ higher score

Suppose the retriever had ranked the three relevant chunks first: $v = (1, 1, 1, 0, 0)$. Then Precision@ k at the relevant positions is $1/1, 2/2, 3/3 = 1.0, 1.0, 1.0$, giving context precision = 1.0. *Exactly the same chunks were retrieved* — only the order changed — yet the score rose from 0.81 to 1.0. The metric is measuring ranking quality, not mere retrieval, which is precisely what a position-sensitive LLM cares about.

4 Reading the score

- **Low context precision** $\Rightarrow$ relevant chunks exist in the context but are ranked below noise. Fix the *ranking*: add a re-ranker (Track 3.3), tune hybrid fusion (3.2), or raise the embedding quality.
- **It says nothing about completeness**: a retriever can have perfect context precision while *missing* a needed chunk entirely — that gap is context recall (#28). Precision and recall must be read together.
- **Downstream effect**: poor context precision wastes the LLM’s limited attention (and context window) on irrelevant text, which can itself induce hallucination or distraction even when the right chunk is present.

Equation cheat-sheet

$$\text{Context Precision} = \frac{\sum_k \text{Precision@}k \cdot v_k}{\# \text{relevant}}, \quad \text{Precision@}k = \frac{\sum_{j \leq k} v_j}{k}$$

rank-aware: rewards relevant-chunks-first; low $\Rightarrow$ add a re-ranker (3.3)

Appendix A · Reference implementation

```
def context_precision(v):
    # v: 1/0 relevance per retrieved chunk
    precs, hits = [], 0
    for k, rel in enumerate(v, start=1):
        if rel:
            hits += 1
            precs.append(hits / k)
    # precision@k at this relevant position
    return sum(precs) / len(precs) if precs else 0.0
```

```
context_precision([1, 0, 1, 1, 0])      # (1.0 + 0.6667 + 0.75)/3 = 0.8056
context_precision([1, 1, 1, 0, 0])      # 1.0   (same chunks, ranked first)
```

Internal learning material. Numbers reproducible from the appendix code. v1.0